📘

SMAPS

Author: guo xiaopang

Abstract

This document presents **SMAPS — Structured Multi-Agent Prompt System**, an engineering-oriented architecture for constructing modular, controllable, reusable, routable, and fault-tolerant prompts. The core thesis is that modern prompts are no longer merely natural-language instructions; in complex generative workflows, they increasingly function as a quasi-programming interface. SMAPS formalizes this shift through a Prompt Design System (PDS), multi-agent FlowChain orchestration, semantic guardrails, modular decomposition, routing logic, stability control, semantic encapsulation, and fallback mechanisms.

The document consolidates five related components into a single white-paper-style specification:

1. SMAPS white paper
2. Prompt Card DSL syntax specification
3. PDS decoupling design guide
4. Semantic encapsulation model
5. Segmentation and semantic anchor framework

Part I — SMAPS White Paper

1. Introduction

From 2023 to 2025, prompts evolved from simple natural-language instructions into a form of **quasi-programming language**. In advanced generative workflows, prompts are expected not only to describe intent, but also to manage structure, constraints, routing, validation, and recovery behavior.

The architecture proposed in this document belongs to a new generation of engineered prompt systems:

> Modular Prompt + Multi-Agent Orchestration + Stability Control + DSL Card Format

This architecture is formally defined as:

SMAPS — Structured Multi-Agent Prompt System

SMAPS is a prompt engineering architecture designed to be:

- Composable
- Reusable
- Controllable
- Routable
- Equipped with fallback mechanisms

Its purpose is to transform prompt writing from an ad hoc textual practice into a structured engineering discipline.

---

2. Background: Why Prompt Engineering Requires System Architecture

Traditional prompts expose several persistent limitations:

- Outputs are difficult to control.
- Models are prone to semantic drift.
- Minor topic changes can cause structural collapse.
- Visual features such as composition, lighting, topology, and face identity are unstable.
- Prompts lack modularity.
- Context boundaries are often unclear.
- Execution paths are not explicitly defined.
- Fallback mechanisms are absent.
- Reuse is limited.

- Larger prompt structures are more likely to collapse.

SMAPS is designed to address these problems systematically by introducing modular decomposition, semantic boundaries, routing logic, stability control, and explicit exception handling into the prompt structure.

---

3. Core Concepts of SMAPS

SMAPS is organized around six core pillars.

3.1 Prompt Cards

A prompt is transformed into a visualizable and reusable “component” composed of UI-like structure and explicit rules.

3.2 Modules

Prompt capabilities are decomposed into independent modules, such as:

- Lighting modules
- Composition modules
- Topology modules
- Identity-lock modules
- Aesthetic modules
- Safety modules

3.3 Multi-Agent FlowChain

The system follows a staged process:

1. Analyze input
2. Route the task
3. Invoke relevant modules
4. Validate execution
5. Generate output
6. Trigger fallback when necessary

3.4 Semantic Guardrails

Guardrails define strict semantic boundaries, such as:

- No cartoonization
- No scene repainting
- No clothing replacement
- No identity drift
- No topology damage

3.5 Logical Control Layer

SMAPS introduces a prompt-level control layer similar to:

```
IF / ELSE / SWITCH / PRIORITY
```

This layer enables conditional behavior within prompt execution.

3.6 Fallback System

When generation becomes abnormal, the system automatically switches to a more stable strategy.

The overall evolution can be summarized as:

> Prompt engineering → Prompt architecture engineering → Prompt language system / DSL

---

4. Prompt Card System: PDS v1.0

PDS v1.0 is compatible with the existing card-based prompt format. Its basic structure is:

```
Role
Type
Context
MCP / Multi-Agent Process Control
Task
Output
Fallback
```

This version is characterized by:

- Minimal structure

- Composability
- Reusability
- Focus on stability and expression

PDS v1.0 is suitable for daily prompt writing where structure is needed but full engineering formalization is not yet required.

---

5. Prompt Design System: PDS v2.0 Extended Edition

PDS v2.0 is the formal engineering version of the Prompt Design System. It introduces a complete syntax layer and modular organization.

```
meta                — purpose / model / mode / version
role                — role definition
type                — output type
context             — context / semantic anchors
guardrails          — constraint rules
modules             — modular components
routing             — conditional routing
task                — execution instructions
stability-control   — drift control
output              — format control
fallback            — fallback system
```

Each module can be upgraded independently. Examples include:

- `modules.lighting`
- `modules.topology`
- `modules.identity-lock`
- `modules.composition`
- `modules.aesthetic`
- `modules.safety`

At this stage, the prompt system begins to acquire **framework-level maintainability**.

---

6. Multi-Agent FlowChain Architecture

The SMAPS FlowChain can be represented as follows:

```
[Role Init]
      ↓
[Context Loader]
      ↓
[Guardrails Activation]
      ↓
[Scene / Face Analyzer Agent]
      ↓
[Routing Engine — determine execution path]
      ↓
[Modules Pipeline — invoke lighting / composition / topology modules]
      ↓
[Stability Control — detect semantic drift]
      ↓
[Final Output Renderer]
      ↓
[Fallback Engine — trigger when necessary]
```

The major functional roles are:

- **Analyzer**: extracts relevant information from the input.
- **Router**: determines the appropriate execution path.
- **Modules**: perform specialized execution tasks.
- **Stability layer**: reviews whether the result has drifted.
- **Fallback layer**: recovers from abnormal output.

By embedding this pipeline into the prompt structure, SMAPS gives prompts engineering properties such as process flow, conditional routing, validation, and recovery.

---

7. Modular Design Logic

SMAPS currently organizes modules into four levels.

7.1 Basic Modules

- Lighting
- Composition
- Color
- Topology lock
- Identity lock

7.2 Scene Modules

- Indoor / outdoor
- Snow
- Rain
- Season
- Environmental elements such as falling leaves, moisture, and fog

7.3 Aesthetic Modules

- Chinese lighting
- Eastern aesthetics
- Gucci style
- Polarized-light photography
- CCD / retro aesthetics

7.4 Constraint Modules

- No cartoonization
- No recomposition
- No clothing change
- No scene replacement

This modular structure makes prompts easier to combine, maintain, extend, and debug.

---

8. Stability and Drift Control

Common stability strategies include:

- `identity-lock`: preserves facial topology.
- `topology-lock`: preserves body pose and scene topology.
- `anti-cartoon`: prevents cartoonization.
- `composition-lock`: preserves framing and spatial arrangement.
- `color-temperature-lock`: prevents uncontrolled color-temperature drift.

Stability control consists of three layers:

1. **Pre-Constraints**

   Constraints are declared before execution.

2. **In-Process Check**

   The system checks whether the generation process deviates from the intended semantic boundary.

3. **Post-Execution Correction**

   The system corrects abnormal output after generation.

This structure is essential because LLM and generative-model outputs are probabilistic and inherently difficult to control.

---

9. Fallback System

SMAPS makes exception handling explicit inside the prompt structure.

Examples:

- If cartoonization appears, force the output back to photorealistic mode.
- If the scene is replaced, force recovery to the original scene topology.
- If falling leaves become too dense, automatically reduce density.
- If composition drifts, return to “Stable Composition v3.0.”

This is equivalent to the following prompt-level control structure:

```
try → check → catch → fallback
```

The fallback mechanism is rarely formalized in ordinary prompt writing, but it is essential for complex prompt systems.

---

10. Routing Engine

Routing defines conditional execution paths. Common routing instructions include:

- Indoor → use Indoor Lighting
- Outdoor → use Outdoor Lighting

- Daytime → use Daytime module
- Night → use Night / Horror module
- If human proportions are unbalanced → adjust composition module
- If a child is detected → use Cute module
- If no hands are detected → apply nearest-object topology plushification

Routing gives prompts a **conditional logic system** at the prompt level.

---

11. Practical Example

The following template illustrates a reusable PDS-style prompt card:

```yaml
meta:
  version: "PDS v2.0"
  target_model: "SeeDream"
  mode: "hybrid"
  purpose: "Stable lighting + Chinese aesthetic rhythm + to
pology preservation"

role: "Eastern lighting and photographic reconstruction spe
cialist"

type: "Cinematic · Chinese Aesthetic · Scene-Preserved"

context:
  - Preserve topology
  - Maintain subject identity
  - Do not allow cartoonization

guardrails:
  anti_cartoon: true
  anti_scene_replace: true
  anti_clothes_change: true

modules:
  lighting: "Chinese side light · qi atmosphere · layered c
ontrast ratio"
```

```
  composition: "rule of thirds · depth-of-field preservatio
n · stable perspective"
  identity: "facial topology preservation"
  environment: "blue-green tone · controllable mist"

routing:
  indoor: use modules.lighting.indoor
  outdoor: use modules.lighting.outdoor
  season.autumn: enable modules.environment.falling_leaves

task:
  - Repair lighting
  - Balance exposure
  - Strengthen Chinese aesthetic atmosphere

stability-control:
  - lock_color
  - lock_composition

output:
  format: "YAML"
  length: "short"

fallback:
  - if drift: use stable_v3.0
```

This structure already demonstrates the characteristics of a prompt language.

12. Comparison with Existing Industry Solutions

Model / Framework	Modular	Multi-Agent	Routing	Fallback
Ordinary Prompt	❌	❌	❌	❌
GPT Script	✔️	Partial	❌	❌
LangChain	✔️	✔️	Partial	❌
AutoGen	✔️	✔️	✔️	❌

Model / Framework	Modular	Multi-Agent	Routing	Fallback
SMAPS	✔️	✔️	✔️	✔️ **— unique**

The defining characteristic of SMAPS is that it combines **modularity, multi-agent workflow, routing, and fallback** within one coherent prompt architecture.

---

13. Future Directions

13.1 Dedicated Prompt DSL

A future DSL may define reusable prompt modules directly:

```
define module Lighting(ChineseAesthetic):
side_light(45deg)
qi_rhyme(enhance)
```

13.2 Prompt Compiler

Prompt cards can be compiled into complete executable prompt structures.

13.3 Graphical Orchestration

Prompt FlowChain can be visualized as a node-based orchestration system similar to LangGraph.

13.4 Open-Source Release

The combination of **PDS v2.0 + SMAPS architecture** can become a systematic solution for the Prompt Design System direction.

Part II — Prompt Card DSL Syntax Specification

1. DSL Overview

PDS, or Prompt Design System, is a structured prompt language designed to be:

- Readable

- Extensible
- Composable
- Routable
- Equipped with fallback mechanisms

The prompt card is the core organizational unit of PDS.

---

2. Core Syntax

2.1 `meta`

The `meta` field defines the metadata of the prompt card. It includes:

- Target model
- Execution mode, such as diffusion, hybrid, or autoregressive
- Version
- System purpose
- Generation goal
- Use case
- Upstream and downstream module interfaces

It functions as the prompt's project definition and configuration file.

```yaml
meta:
  version: "2.0"
  target_model: "SeeDream v4"
  mode: "hybrid"
  purpose: >
    Reconstruct Chinese-style lighting for a portrait photo
while preserving topology,
    identity, and clothing consistency.
  author: "alex"
```

Rules:

- `meta` must appear first.
- `version` should follow semantic versioning.

- `purpose` should describe the intended outcome, not the action instruction.
- Model parameters such as denoising strength should not be written here because they are not prompt content.

2.2 `role`

The `role` field defines the professional identity, capability scope, and style baseline that the AI should assume.

It acts as the prompt's primary persona anchor.

```
role: >
  You are an Eastern aesthetic photography reconstruction s
pecialist,
  proficient in Chinese lighting, topology locking, and rea
listic human texture.
```

Rules:

- Use the structure "You are..." followed by "proficient in...".
- Use this field to control style and capability orientation.
- Use this field to maintain output-logic consistency.
- Do not include task content; task content belongs in `task`.

2.3 `type`

The `type` field defines the output category, task category, or style category.

Examples:

- Cinematic
- Photographic Reconstruction
- Chinese Aesthetic
- HDR Light Solver
- Texture Preserver

It functions as a domain switch for the model.

```
type: "Type description, such as photography, video, or sty
le"
```

Rules:

- Use concise phrase combinations.
- Do not include task verbs.
- Do not include constraints; constraints belong in `guardrails`.

2.4 `context`

The `context` field injects input semantics, environmental conditions, and immutable boundary conditions into the task.

It belongs to the prompt's Input Layer.

Typical content includes:

- Scene topology preservation
- Identity preservation
- Background immutability
- Current lighting or color temperature
- Current composition information, such as "half-body portrait"

```
context:
  - The identity of the person in the original image must r
emain consistent.
  - The real topology of the scene must be strictly preserv
ed.
  - The background must not be replaced.
  - The light source direction must not be changed.
  - The subject composition is a half-body portrait; full-b
ody generation is not allowed.
```

Rules:

- `context` should state facts and non-violable truths.
- Do not write aspirational statements such as "hope to achieve."

- Keep the description clear but not verbose.
- `context` is the upstream semantic source for `guardrails`.

2.5 `guardrails`

The `guardrails` field defines semantic boundaries that are absolutely forbidden, must be avoided, or must not be crossed.

Examples:

- No cartoonization
- No automatic hairstyle change
- No automatic clothing change
- No automatic scene replacement
- No topology damage
- No background repainting triggered by blur

This field functions as the prompt's safety layer and boundary layer.

```yaml
guardrails:
  anti_cartoon: true
  anti_scene_replace: true
  anti_clothes_change: true
  preserve_topology: true
  preserve_identity: true
```

Rules:

- `guardrails` are strong rules.
- They are used to suppress unwanted automatic inference.
- They provide the main guarantee for task stability.

2.6 `modules`

The `modules` field contains reusable processing submodules.

Examples:

- `lighting`

- `topology`
- `identity-lock`
- `composition`
- `environment`
- `aesthetic`

Modules are the key assets that enable reuse, composition, and system upgrades.

```yaml
modules:
  lighting:
    mode: "chinese_side_light"
    ratio: "3:1"
  composition:
    lock: true
  identity:
    preserve: true
```

Rules:

- Modules are the system's most important extensible assets.
- Modules may be versioned, such as `lighting_v3.2`.
- The more independent a module is, the easier it is to compose.

2.7 `routing`

The `routing` field determines which module path should be activated based on input conditions.

Examples:

- Indoor → indoor lighting
- Outdoor → outdoor lighting
- Daytime → sunlight module
- Night → night module
- Rainy scene → wet-surface module
- Composition drift → composition stable v2

Routing gives prompts branching logic.

```
routing:
  if indoor: use modules.lighting.indoor
  if outdoor: use modules.lighting.outdoor
  if face_small: enable modules.identity.super_res
```

Rules:

- Avoid unnecessary complexity.
- `routing` acts as the selector for multi-agent behavior.
- In structurally complex tasks, routing can significantly improve success rate.

2.8 `task`

The `task` field defines what the model should do in the current execution.

Examples:

- Repair lighting
- Balance exposure
- Strengthen Chinese aesthetic atmosphere
- Replace hairstyle
- Preserve topology
- Keep composition unchanged

This is the prompt's Action Layer.

```
task:
  - Repair lighting
  - Adjust exposure
  - Strengthen Chinese atmosphere and qi rhythm
  - Preserve subject topology and identity
```

Rules:

- `task` should contain actions.
- Do not write style descriptions here.

- Do not write hard constraints here; hard constraints belong in `guardrails`.

2.9 `stability-control`

The `stability-control` field addresses common collapse problems in complex prompts:

- Composition drift
- Automatic head replacement
- Automatic scene replacement
- Automatic clothing replacement
- Invented lighting
- Random color-temperature changes
- Re-posing

This field functions as an AI safety layer within the prompt architecture.

```yaml
stability-control:
  - lock_identity
  - lock_color_temperature
  - lock_composition
  - anti_drift
```

2.10 `output`

The `output` field controls response format and presentation.

It may define:

- JSON / YAML / Markdown
- Short or detailed output
- Segmented output
- Whether comments are allowed
- Whether the output should be the prompt itself or an image instruction

```yaml
output:
  format: "YAML"
```

```
  length: "short"
  style: "neutral"
```

2.11 `fallback`

The `fallback` field defines recovery behavior when the model drifts, collapses, becomes cartoon-like, corrupts composition, or changes the scene.

It addresses long-term stability problems such as:

- Cartoonization
- Baby-like enlarged heads
- Plastic-looking subjects
- Scene repainting
- Clothing replacement
- Body-pose drift

```
fallback:
  if drift: use stable_v3.0
  if cartoon_detected: enforce photorealistic_mode
  if scene_replace: revert_to_original_topology
```

Rules:

- `fallback` is the prompt's exception-handling mechanism.
- The more complex the prompt is, the more necessary fallback becomes.

3. Overview of PDS Syntax Layers

PDS contains 11 syntax layers. Together, they define identity, type, input, constraints, modules, routing, behavior, stability, output, and recovery.

```
meta → role → type → context → guardrails
→ modules → routing → task → stability → output → fallback
```

Part III — PDS Decoupling Design Guide

1. Why Prompts Must Be Decoupled

Large generative models can be understood as:

Probabilistic Semantic Systems

When multiple tasks are mixed into a single prompt:

- The model infers what it believes the user intends.
- Modules override one another.
- Lighting changes composition.
- Backgrounds are repainted.
- Clothing material changes.
- Falling leaves block faces.
- CCD style shifts skin tone.
- Fog changes color temperature.
- Wetness modules corrupt clothing details.
- Night modules trigger scene repainting.

The practical conclusion is:

> The more complex a prompt becomes, the more likely it is to structurally collapse if it lacks decoupling.

---

2. Three Dimensions of Decoupling

Prompt decoupling in SMAPS/PDS has three layers.

2.1 Functional Decoupling

Functional decoupling ensures that different capabilities do not interfere with one another.

Examples:

- Lighting ≠ composition
- Composition ≠ pose topology
- Falling leaves ≠ background repainting

- Fog ≠ color management
- Skin texture ≠ CCD tone
- Wetness ≠ clothing material

2.2 Semantic Decoupling

Semantic decoupling prevents vague commands from causing unintended cross-module inference.

For example, the command “enhance wetness” may cause the model to infer that:

- Clothing material should change.
- Skin should become glossy.
- The environment should become wet.

A decoupled representation is:

```
wet_surface:
  affect: "specular_only"
clothes:
  preserve_material: true
skin:
  preserve_texture: true
```

This reduces the probability of cross-module inference.

2.3 Stability Decoupling

Stability decoupling prevents:

- Model drift
- Repeated enhancement
- Later steps changing the output of earlier steps

Relevant mechanisms include:

- `lock_identity`
- `lock_color`
- `lock_composition`

- `anti_drift`
- `fallback`

This layer is critical for maintaining consistency in complex prompts.

---

3. PDS Tools for Decoupling

PDS provides four primary tools for decoupling.

3.1 `modules`

Modules separate capabilities. Each module should do only one thing.

3.2 `routing`

Routing controls when each module is activated and prevents incorrect module execution.

3.3 `guardrails`

Guardrails prevent cross-domain semantic pollution caused by automatic inference.

3.4 `fallback`

Fallback repairs abnormalities that remain after module execution.

Together, these components form:

> A layered MVC-like architecture for prompts

---

4. Common Coupling Problems

4.1 Lighting Module → Clothing Material Change

Cause: lighting and material share semantic space.

Decoupling:

```
modules:
  lighting: { apply: "global_only" }
  clothes: { preserve_material: true }
```

4.2 Lighting Enhancement → Scene Repainting

Cause: “reconstruct lighting” may be interpreted as “rerender the scene.”

Decoupling:

```yaml
guardrails:
  anti_scene_replace: true
modules:
  lighting: { apply: "global_only" }
  scene: { preserve_topology: true }
```

4.3 CCD / Retro Filter → Skin Turns Green or Gray

Cause: the tone module is coupled with the skin module.

Decoupling:

```yaml
modules:
  color: { ccd_curve: "soft_low_contrast" }
  skin: { prevent_skin_shift: true }
```

4.4 Fog / Atmosphere Module → Color Temperature or Exposure Drift

Cause: the model associates fog with cold tone or low exposure.

Decoupling:

```yaml
modules:
  environment:
    fog: { density: low }
  color:
    lock_temperature: true
    lock_exposure: true
```

4.5 Falling Leaves → Subject or Face Occlusion

Cause: the model does not infer that the face must remain unobstructed.

Decoupling:

```
modules.environment:
  avoid_face: true
  density: low
```

4.6 Wetness Module → Clothing Material Change

Cause: the model infers that wetness requires material transformation.

Decoupling:

```
wet_surface:
  affect: "specular_only"
clothes:
  preserve_material: true
```

4.7 Night Mode → Scene Replacement

Cause: the model interprets “night” as a different background.

Decoupling:

```
guardrails:
  anti_scene_replace: true
modules:
  lighting:
    type: "night_adjustment"
```

4.8 Human Reconstruction → Facial Proportion Drift

Cause: the model’s beautification logic overrides the instruction.

Decoupling:

```
modules:
  identity:
    strict: true
  face_pose:
    lock: true
```

4.9 Composition Module → Overridden by Lighting or Style

Composition should be treated as a high-priority constraint and locked early.

4.10 Indoor Snow → Outdoor Scene Inference

This problem should be solved through routing.

```
routing:
  if indoor: inject snow_from_one_side
```

4.11 Hairstyle Replacement Module → Identity Drift

Hairstyle replacement must be decoupled through `identity-lock`.

4.12 Tone Module → Monochrome Scene

Guardrails should be added to prevent unintended monochrome transformation.

```
color:
  prevent_monotone: true
```

5. Decoupling Design Strategies

Rule 1: One Module Should Do One Thing

This follows the Single Responsibility Principle.

Incorrect:

```
lighting:
  fix_shadow_and_add_sunset_and_balance_color
```

Correct:

```
lighting_fix
lighting_sunset
color_balance
```

Rule 2: Modules Must Not Share Vague Semantics

“Enhance realism” is a high-risk phrase. It should be decomposed into:

- Lock topology
- Add micro-texture
- Balance color temperature
- Increase contrast

Rule 3: Strongly Coupled Semantics Must Be Split

For example, “fog + tone + atmosphere” should be separated into:

- `fog`
- `tone`
- `atmosphere`

Rule 4: Stability Modules Always Have the Highest Priority

The following modules must be placed at the top of the system:

- `identity-lock`
- `composition-lock`
- `topology-lock`

Rule 5: Environment-Dependent Capabilities Must Be Triggered by Routing

Examples include:

- Snow
- Fog
- Falling leaves
- Wetness
- Night scene

These should be activated through routing:

```
routing:
  if indoor: use modules.snow.side_wind
  if outdoor: use modules.snow.full_space
```

Rule 6: Any Module That May Damage Topology Must Have Fallback

Example:

```
fallback:
  if drift: use stable_v3.1
```

6. Module Dependency Graph

```
identity-lock
     ↑
topology-lock
     ↑
composition
     ↑
lighting
     ↑
color
     ↑
environment
     ↑
aesthetic
```

The required dependency order is:

1. **Identity**: the bottom layer; immutable.
2. **Topology**: immutable.
3. **Composition**: strong constraint.
4. **Lighting**: modifiable.
5. **Color**: modifiable.
6. **Environment**: weak constraint.
7. **Aesthetic**: lowest priority.

7. Module Priority

Priority	Module	Description
P0	identity / topology	Never overridden
P1	composition	Controls framing and must not be overridden by lighting
P2	stability	Maintains consistency
P3	lighting	Main degree of freedom
P4	color	Secondary degree of freedom
P5	environment	Light variation
P6	aesthetic	Lowest priority and easily overridden

8. Communication Rules Between Modules

To avoid contamination, modules should:

- Avoid sharing fields with identical names.
- Avoid vague verbs such as `enhance` and `improve`.
- Use precise physical parameters such as `ratio` and `density`.
- Use routing for conditional triggering.
- Use guardrails to limit automatic inference.

9. Prompt Refactoring Practice

9.1 Coupled Prompt

```
Enhance Chinese lighting, add fog to the scene, preserve the background,
and give the person a wet feeling, but do not change the clothing,
keep the skin texture, and make it natural.
```

Potential errors include:

- Fog changes color temperature.
- Wetness changes clothing material.
- Background is repainted.

9.2 Decoupled PDS Version

```yaml
modules:
  lighting: { chinese_side_light }
  fog: { density: low, color_neutral: true }
  wet_surface: { affect: specular_only }
  clothes: { preserve_material: true }
  skin: { preserve_texture: true }
guardrails:
  anti_scene_replace: true
  anti_clothes_change: true
routing:
  if indoor: use modules.fog.edge_soft
stability:
  - lock_identity
  - lock_composition
```

---

10. Future Extensions: Automated Decoupling and Prompt Compiler

A future **Prompt Compiler** can be constructed as follows:

- **Input**: natural-language description
- **Output**: standardized, decoupled YAML prompt card based on PDS

Applicable scenarios include:

- Scene generation
- Multi-agent orchestration
- Design systems
- Image generation
- Image editing systems
- UI DSL generation

Part IV — PDS Module Library v1.0

1. Identity and Face Modules

Identity and face modules preserve facial features, skin texture, and identity. They represent the highest-priority module group, P0.

1.1 `identity_lock`

```yaml
identity_lock:
  strict: true
  match_ratio: 0.95
  preserve_age: true
  preserve_gender: true
```

1.2 `face_topology_lock`

```yaml
face_topology_lock:
  preserve_eye_shape: true
  preserve_nose: true
  preserve_lip_shape: true
  preserve_eye_spacing: true
```

1.3 `skin_texture_preserve`

```yaml
skin_texture_preserve:
  pores: "visible_natural"
  micro_shadows: true
  vellus_hair: true
```

1.4 `body_shape_preserve`

```yaml
body_shape_preserve:
  strict: true
  no_slimming: true
  no_enhance: true
```

1.5 `hair_preserve`

```yaml
hair_preserve:
  keep_style: true
  keep_length: true
  keep_color: true
```

2. Composition Modules

Composition modules are P1 high-priority modules used to constrain the image frame.

2.1 `composition_lock`

```yaml
composition_lock:
  framing: "half-body"
  angle: "front"
  camera_distance: "medium"
  forbid_full_body: true
```

2.2 `rule_of_thirds`

```yaml
rule_of_thirds:
  subject_position: "upper_center"
```

2.3 `portrait_composition`

```yaml
portrait_composition:
  crop: "shoulder_head"
  avoid_tilt: true
```

2.4 `avoid_mirroring`

```yaml
avoid_mirroring:
  prevent_left_right_flip: true
```

3. Scene and Body Topology Modules

Topology modules preserve pose, background position, and scene space.

3.1 `scene_topology_lock`

```yaml
scene_topology_lock:
  preserve_geometry: true
  preserve_background_structure: true
  no_scene_replacement: true
```

3.2 `pose_lock`

```yaml
pose_lock:
  strict: true
  no_new_pose: true
```

3.3 `background_preserve`

```yaml
background_preserve:
  strict: true
  color_shift_allowed: false
```

---

4. Lighting Modules

Lighting is the most complex module family and belongs to P3.

4.1 `chinese_side_light`

```yaml
chinese_side_light:
  direction: "45_left"
  softness: "medium_soft"
  contrast_ratio: "3:1"
  qi_rhyme: "enhanced"
```

4.2 `cinematic_soft_light`

```yaml
cinematic_soft_light:
  softness: high
```

```
  haze: low
  backlight: mild
```

4.3 `highlight_repair`

```
highlight_repair:
  restore_clipped_area: true
  preserve_skin_texture: true
```

4.4 `shadow_rebuild`

```
shadow_rebuild:
  face_shadow: "natural"
  body_shadow: "soft"
  maintain_direction: true
```

---

5. Color and Tone Modules

Color and tone modules address CCD effects, warm/cool color shifts, weather-related tone changes, and exposure problems.

5.1 `color_balance`

```
color_balance:
  neutral_gray: true
  temperature_lock: true
```

5.2 `tone_autumn`

```
tone_autumn:
  base_color: "warm_yellow"
  contrast: mild
  saturation: moderate
```

5.3 `ccd_vintage`

```yaml
ccd_vintage:
  tone_curve: "soft_low_contrast"
  chroma_shift_blue: -5
  prevent_skin_shift: true
```

5.4 `horror_night`

```yaml
horror_night:
  temperature: 4200K
  cyan_shift: +3
  shadow_depth: high
```

---

6. Environment Modules

Environment modules add falling leaves, snow, fog, wetness, rain, and related environmental logic.

6.1 `falling_leaves`

```yaml
falling_leaves:
  leaf_type: "ginkgo"
  density: low
  avoid_face: true
  avoid_camera_center: true
```

6.2 `snow_module`

```yaml
snow_module:
  flake_shape: "hexagon"
  density: medium
  motion_blur: mild
  accumulate_logic: true
```

6.3 `fog_module`

```yaml
fog_module:
  density: low
  color_neutral: true
  avoid_face: true
```

6.4 `wet_surface`

```yaml
wet_surface:
  affect: "specular_only"
  avoid_clothes_material_change: true
```

6.5 `rain_module`

```yaml
rain_module:
  raindrop_density: low
  ambient_wetness: true
  adjust_reflection: mild
```

---

7. Aesthetic Modules

Aesthetic modules provide advanced style control.

7.1 `chinese_aesthetic`

```yaml
chinese_aesthetic:
  qi_rhyme: "high"
  air_layer_depth: "multi"
  ink_softness: mild
```

7.2 `gucci_oriental`

```yaml
gucci_oriental:
  luxury_color: "deep_red+gold"
  asian_face_proportion: true
  no_logo: true
```

7.3 `angel_cinematic`

```yaml
angel_cinematic:
  wing_material: "feather_film_grade"
  feather_fall: "soft"
```

---

8. Special Effects Modules

Special effects modules increase dramatic intensity.

8.1 `fire_particles`

```yaml
fire_particles:
  glow: mild
  motion_blur: medium
  color: "orange_red"
```

8.2 `dust_motion`

```yaml
dust_motion:
  density: low
  highlight_edge: true
```

8.3 `polarized_light`

```yaml
polarized_light:
  reflection_suppression: high
  color_depth: enhanced
```

---

9. Stability Modules

Stability modules maintain prompt-output consistency.

9.1 `identity_stability`

```yaml
identity_stability:
  lock_feature: true
```

```
  prevent_facial_reconstruction: true
```

9.2 `composition_stability`

```
composition_stability:
  prevent_auto_zoom: true
  prevent_recenter: true
```

9.3 `anti_drift`

```
anti_drift:
  before_after_match: true
  prevent_cumulative_effects: true
```

9.4 `anti_cartoon`

```
anti_cartoon:
  skin_realism: true
  forbid_smooth_plastic: true
```

---

10. Modules to Be Extended

This section is reserved for future module expansion.

Part V — Semantic Encapsulation

1. Why Prompts Need Semantic Encapsulation

When prompts involve multiple simultaneous elements — such as lighting, composition, topology, wetness, qi rhythm, color, snow, and falling leaves — models may produce several failure modes:

- Modules override one another.
- The model invents missing intent.
- Logical conflicts cause collapse.

- Color affects lighting.
- Lighting affects composition.
- Composition affects pose.
- CCD modules contaminate skin tone.
- Wetness modules change clothing material.
- Scenes are repainted.
- Composition drifts.
- Identity changes.

These problems share one root cause:

> The model does not know the intended semantic boundaries.

More specifically:

> The model does not know which semantic areas should be isolated, which areas may interact, and which instructions must remain locked.

Therefore, SMAPS introduces **Semantic Encapsulation**.

---

2. Definition of Semantic Encapsulation

Semantic encapsulation means encapsulating a semantic unit — such as a capability, effect, style, or rule — inside an independent semantic container so that it affects only its own region and does not interfere with other modules.

It is analogous to programming concepts such as:

- Function
- Class
- Scope
- Interface
- Namespace

In prompt architecture, semantic encapsulation helps the model understand:

- Which semantic unit belongs to which module
- Which instructions must not cross module boundaries

- Which logic is locked
- Which effects must not interfere with one another

3. Core Value of Semantic Encapsulation

3.1 Preventing Semantic Contamination

Example:

A wetness module may cause the model to change clothing material. Encapsulation isolates the wetness semantics:

```
wet_surface:
  affect: specular_only
```

This reduces the probability that wetness semantics will diffuse into clothing, skin, or background domains.

3.2 Improving Stability and Determinism

Multi-module tasks collapse easily when semantic domains are mixed. The solution is:

> Split instructions, encapsulate them independently, and prevent cross-domain interference.

3.3 Supporting Composable Prompt Engineering

Encapsulation allows modules to be combined like building blocks:

```
lighting.chinese_side
fog.soft_low
env.leaves.low_density
skin.preserve
ccd.soft_curve
```

This is the programmable capability of prompt architecture.

4. Types of Semantic Encapsulation

4.1 Functional Encapsulation

Examples:

- Lighting
- Falling leaves
- Wetness
- CCD

4.2 Constraint Encapsulation

Examples:

- `identity_lock`
- `scene_lock`
- `composition_lock`

4.3 Aesthetic Encapsulation

Examples:

- `chinese_aesthetic`
- `gucci_oriental`

4.4 Stability Encapsulation

Examples:

- `anti_drift`
- `anti_cartoon`

Each encapsulation type should modify only its own semantic domain.

---

5. Semantic Contamination

Semantic contamination is the root cause of many prompt collapses.

5.1 Visual Attribute Contamination

Examples:

- Lighting changes color.
- Falling leaves change composition.

- Fog changes exposure or color temperature.

5.2 Identity Contamination

Examples:

- Hairstyle modification triggers face repainting.
- CCD shifts skin tone toward green or gray.

5.3 Task Overflow

Examples:

- “Repair lighting” causes background replacement.
- “Make it night-like” causes the model to generate a new night scene.

All of these can be mitigated through semantic encapsulation.

---

6. Semantic Encapsulation Mechanisms in PDS

PDS implements semantic encapsulation through four mechanisms.

6.1 `modules` as Semantic Containers

Each module defines an independent semantic space.

6.2 `guardrails` as Semantic Fences

Guardrails prevent module semantics from diffusing outward.

6.3 `routing` as Semantic Entry Control

Routing activates the correct module only under the correct condition.

6.4 `fallback` as Error Recovery

Fallback returns the system to a stable strategy when encapsulation fails.

---

7. Writing Templates for Semantic Encapsulation

7.1 Single-Function Encapsulation

```
lighting_chinese:
  direction: "45_left"
```

```
  softness: "medium"
  qi_rhyme: "enhanced"
```

7.2 Constraint Encapsulation

```yaml
skin_preserve:
  pores: natural
  avoid_beautify: true
```

7.3 Conditional Encapsulation

```yaml
fog_edge_soft:
  density: low
  color_shift: none
  avoid_face: true
```

7.4 Combined Encapsulation

```
lighting_cinematic
skin_preserve
composition_lock
environment_autumn_low
```

---

8. Case Study

8.1 Unencapsulated Prompt

```
Enhance lighting, add fog to the environment, keep skin texture natural,
and do not change the background.
```

This prompt is likely to collapse because lighting, fog, skin, and background constraints are mixed in one semantic space.

8.2 Encapsulated Prompt

```yaml
modules:
  lighting.cinematic_soft
  fog.soft_low
  skin.preserve
  scene.lock

guardrails:
  no_scene_replace: true
  anti_cartoon: true
```

The encapsulated version separates functional modules and reinforces boundaries through guardrails.

---

9. Future Extensions

Semantic encapsulation can evolve into:

- A module library
- A prompt compiler
- A visual orchestration tool
- A versioned review system for reusable prompt structures

Part VI — Segmentation and Semantic Anchors

1. Definition

In complex prompts, **segmentation + semantic anchors** is a key structural method.

```
Segmentation = structural layering
Semantic Anchor = critical semantics that must not be dropped or ignored
```

This structure serves several purposes:

- It divides complex tasks into clear parts.

- It tells the model that each segment is an independent instruction block.
- It prevents critical requirements from being ignored.
- It protects priority from being overridden.
- It strengthens necessary semantics.

---

2. Why Segmentation Is Necessary

Complex prompts often suffer from the following problems:

- The model ignores specific requirements.
- Two logical sections override each other.
- The model chooses what it considers most important.
- The latter part of a long paragraph is truncated or weakened.

Segmentation improves:

- Model comprehension
- Semantic isolation
- Step-by-step reading
- Modular instruction execution
- Resistance to override
- Semantic anchoring

---

3. Types of Semantic Anchors

3.1 Hard Anchor

A hard anchor must not be ignored and must be enforced.

Examples:

- `preserve_topology`
- `preserve_identity`
- `anti_cartoon`
- `no_scene_replace`

Hard anchors have the highest priority, typically P0–P1.

3.2 Soft Anchor

A soft anchor should be followed, but optimization is allowed.

Examples:

- `qi_rhyme_enhanced`
- `cinematic_soft_light`
- `warm_yellow_autumn`

3.3 Conditional Anchor

A conditional anchor is activated by routing.

Example:

```
if indoor: enable snow_from_side
```

4. Writing Semantic Anchors in PDS

Hard anchors should be placed in `guardrails` or `context`, not in `modules`.

```yaml
guardrails:
  preserve_topology: true
  preserve_identity: true
  anti_cartoon: true
```

Soft anchors may be placed in `modules`.

```yaml
modules:
  lighting:
    qi_rhyme: enhanced
```

Conditional anchors should be placed in `routing`.

```yaml
routing:
  if indoor: use snow.side_direction
```

5. Priority of Semantic Anchors

Priority	Anchor Type	Example
P0	Hard anchor	`preserve_identity`
P1	Hard anchor	`preserve_topology`
P2	Soft anchor	`chinese_qi_rhyme`
P3	Conditional anchor	`indoor_snow`
P4	Aesthetic anchor	`gucci_oriental`

6. Standard Segmented Template

```
# === 1. purpose ===
Explain the fundamental objective of the task.
Do not write instructions or parameters here.

# === 2. context ===
State facts that must not change, such as composition, pose, and topology.

# === 3. anchors ===
Hard anchors: immutable constraints
Soft anchors: style direction
Conditional anchors: activated by routing

# === 4. modules ===
Lighting / color / identity lock / falling leaves / environment, etc.

# === 5. stability ===
identity-lock
composition-lock
anti-drift

# === 6. output ===
Specify the output structure, such as YAML or JSON.
```

7. Example

7.1 Unsegmented Prompt

```
Keep the subject topology unchanged, enhance lighting, prevent cartoonization,
and keep skin texture natural.
```

This version is easy for the model to partially ignore because all instructions are mixed together.

7.2 Segmented Prompt

```
# === purpose ===
Reconstruct realistic lighting while preserving topology and identity.

# === anchors (hard) ===
preserve_identity: true
preserve_topology: true
anti_cartoon: true

# === modules ===
lighting.chinese_side
skin.preserve

# === stability ===
lock_identity
lock_composition
```

The segmented version improves semantic boundaries, priority clarity, and execution order.

---

8. Application in Complex Image Tasks

The segmentation and semantic-anchor structure improves stability in complex image tasks such as:

- Portrait reconstruction
- Lighting repair
- Environmental diffusion, including fog, rain, and snow
- Composition changes
- Color management, including CCD, night scene, and autumn tone
- Style mixing, such as Gucci × Chinese style
- Identity and topology stability
- Automatic judgment of environment, season, and object position
- Multi-agent orchestration

For the common problem of sudden model collapse in complex prompts, segmentation and semantic anchoring are core stabilization methods.

Conclusion

SMAPS reframes prompt engineering as a structured systems discipline. Its central contribution is not a single prompt template, but an architecture that combines modular decomposition, semantic boundaries, multi-agent orchestration, routing, stability control, fallback, semantic encapsulation, and semantic anchors.

Under this architecture, prompts become maintainable, extensible, reviewable, and more stable. PDS provides the syntax layer; FlowChain provides the execution layer; guardrails and anchors provide the constraint layer; modules provide the compositional layer; routing provides the decision layer; and fallback provides the recovery layer.

Together, these components form a practical foundation for building prompt systems that behave less like isolated instructions and more like engineered semantic runtimes.